Funzioni di ordine superiore

L06 · Programmazione funzionale · OCaml

Mauro Leoncini

Obiettivi

- Le funzioni sono valori, e portano con sé un ambiente
- Currying: perché si scrive `f x y` e non `f(x, y)`
- Che cosa un tipo polimorfo vi dice che la funzione **non può fare**
- `map`, `filter`, `fold`: la ricetta di L04 diventa un valore

L'avete già vista, a L03

```
let x = 3
let f () = x + 1
let x = 100

f ()      (* 4 *)
```

A L03 la spiegazione era dal lato di `let`: il secondo legame nasconde il primo, non lo modifica.

Oggi la stessa cosa dall'altro lato: `f` non è soltanto codice, è **codice più l'ambiente in cui è stata scritta**, e quell'ambiente se lo porta appresso.

Il nome di questa coppia è **chiusura**.

Una funzione che ne restituisce un'altra

```
let times k = fun n -> n * k

let double = times 2
let triple = times 3
```

Lo stesso identico codice, `fun n -> n * k`, con due ambienti diversi.

Il tranello che in OCaml non esiste

```
functions = []
for i in range(3):
    functions.append(lambda n: n + i)

[f(10) for f in functions]      # [12, 12, 12]
```

La lambda cattura **la variabile** `i`, non il suo valore, e alla fine del ciclo `i` vale 2 per tutte e tre.

In OCaml non può succedere: un legame non cambia dopo essere stato creato, quindi catturare il legame e catturare il valore sono la stessa cosa.

Perché `f x y` e non `f(x, y)`

```
let add a b = a + b
```

```
add : int -> int -> int
    = int -> (int -> int)
```

La freccia associa a **destra**: `add` non è una funzione di due argomenti, è una funzione di **un** argomento che restituisce una funzione di un argomento.

Quindi l'applicazione associa a **sinistra**: `add 10 5` è `(add 10) 5`.

Applicazione parziale

```
let add_ten = add 10                    (* int -> int *)

let evens = List.filter (fun n -> n mod 2 = 0)
let sum_all = List.fold_left ( + ) 0
```

`evens` e `sum_all` sono definite **senza nominare la lista**.

Non è un vezzo di stile: sono funzioni già esistenti a cui è stato dato qualche argomento, e il risultato ha un nome.

La domanda di L05, esercizio 5

Perché `'a albero` e non `int albero`?

```
length : 'a list -> int
```

Il tipo non dice solo che accetta liste di qualunque cosa. Dice qualcosa di molto più forte:

> questa funzione **non può guardare** gli elementi

Non sa se sono numeri, non può confrontarli, non può sommarli. L'unica cosa a cui ha accesso è la **struttura** della lista.

Un teorema che si legge nel tipo

Quante funzioni hanno questo tipo?

```
'a -> 'a
```

Una sola. Ricevuto un valore di un tipo di cui non si sa nulla, non lo si può ispezionare, modificare, confrontare, né fabbricarne un altro: l'unica cosa restituibile è quella arrivata.

Nessuno ha letto il corpo della funzione.

Il caso utile

```
rearrange : 'a list -> 'a list
```

Qualunque funzione con questo tipo può solo copiare, scartare, duplicare e riordinare gli elementi ricevuti. **Non può inventarne.**

Se nella lista in uscita compare un 7, quel 7 era già in ingresso.

Confrontate con `int list -> int list`, che può fare qualunque cosa.

Il prezzo, e la ragione di oggi

Se `'a` non sa confrontarsi, il confronto deve **arrivare da fuori**:

```
max_by : ('a -> 'a -> bool) -> 'a list -> 'a option
```

Ecco perché esistono le funzioni di ordine superiore, prima ancora della comodità: sono ciò che rende utilizzabile il codice che non conosce i propri dati.

Tre funzioni, una sola forma

```
let rec length = function
  | [] -> 0
  | _ :: r -> 1 + length r

let rec sum = function
  | [] -> 0
  | x :: r -> x + sum r

let rec concat_strings = function
  | [] -> ""
  | s :: r -> s ^ concat_strings r
```

Cambiano due cose sole: il **valore di partenza** e il modo di **combinare**.

La ricetta di L04, diventata un valore

```
let rec fold combine initial = function
  | [] -> initial
  | x :: r -> combine x (fold combine initial r)
```

```
fold : ('a -> 'b -> 'b) -> 'b -> 'a list -> 'b
```

```
let length         l = fold (fun _ acc -> 1 + acc) 0 l
let sum            l = fold ( + ) 0 l
let concat_strings l = fold ( ^ ) "" l
```

Un argomento per costruttore: `[]` dà `initial`, `::` dà `combine`.

E anche `map` e `filter`

```
let map    f l = fold (fun x acc -> f x :: acc) [] l
let filter p l = fold (fun x acc -> if p x then x :: acc else acc) [] l
```

Quindi non sono tre astrazioni: è una sola, con tre usi frequenti che hanno meritato un nome.

Proprietà

Nota: il simbolo `%>` viene definito "di norma" come *forward composition* di funzioni, ovvero la composizione **da sinistra a destra**

Per ogni `f`, `g`, `l`:

```
map f (map g l)     =  map (g %> f) l
length (map f l)    =  length l
length (filter p l) <= length l
```

- oggi le **controlliamo** su una lista
- a **L08** le dimostriamo per induzione, con la solita forma
- a **L09** le fa cercare QCheck su liste generate a caso

C++ vi obbliga a scegliere

```
int k = 2;
auto by_value     = [k](int n)  { return n * k; };   // copia k
auto by_reference = [&k](int n) { return n * k; };   // rinvia a k

k = 3;

by_value(5)     = 10
by_reference(5) = 15
```

Lo stesso tranello di Python, e la sua cura, dentro un solo linguaggio: la differenza è **un carattere**, la `&`.

E la scelta sbagliata non fa rumore

```
std::function<int(int)> factory() {
    int k = 2;
    return [&k](int n) { return n * k; };   // k muore qui
}
```

Il riferimento resta appeso nel vuoto: comportamento indefinito.

Con `g++ -Wall -Wextra`: **nessun avviso**. Compila in silenzio.

Tre linguaggi, tre risposte

Python	non vi fa scegliere, e dà la versione che sorprende
C++	vi obbliga a scegliere, e la scelta sbagliata è indefinita
OCaml	la domanda non si pone: un legame non cambia mai

La terza riga non è una comodità in più: è **una domanda in meno** a cui rispondere, per sempre, in ogni programma.

Python ha preso metà dell'idea

```
[n * 10 for n in l if n % 2 == 0]        # map and filter, in the syntax

from functools import reduce             # the fold, in a module
reduce(lambda acc, n: acc + n, l, 0)
```

In Python 3 `reduce` è stata tolta dalle funzioni predefinite, per scelta esplicita: meno leggibile di un ciclo `for`.

Eppure oggi abbiamo visto che `map` e `filter` sono **casi particolari** di `fold`.

Che cosa portate a casa oggi

- Una chiusura è codice **più** l'ambiente in cui è nata
- `f x y` è `(f x) y`, e da lì l'applicazione parziale
- Un tipo polimorfo è una specifica: dice ciò che la funzione **non può** fare
- `fold` è la ricorsione strutturale con i buchi aperti; `map` e `filter` ne sono casi
- Un nome più specifico di `fold` è una **specifica più forte**, non solo comodità

Al calcolatore

```
lezioni/L06-ordine-superiore/codice/
  chiusure.ml            double e triple, stesso codice, ambienti diversi
  currying.ml            i tipi, verificati con ocamlc -i
  parametricita.ml       che cosa 'a -> 'a vi impedisce
  mappa_filtra_piega.ml  le tre funzioni, la forma comune, le leggi
  ordine.cpp             10 e 15: la differenza è una &
  ordine.py              il ciclo che stampa [12, 12, 12]
  esercizi.ml            cinque esercizi, il terzo e il quarto senza codice
```

Riferimenti

- Esercizio da fare prima di L07: il punto 2, `fold_tree`, con il tipo scritto **prima** dell'implementazione
- Domanda aperta, per L07: `fold` costruisce la risposta risalendo. Su una lista di un milione di elementi, che cosa succede?